Section 1: The Problem-Solving Framework

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 1: The Five-Step Problem-Solving Framework

SLIDE 2: Constraints Tell You the Pattern

SLIDE 3: How to Communicate: Brute Force to Optimal

SLIDE 4: Section Cheat Sheet

SLIDE 1: The Five-Step Problem-Solving Framework

Section 1: The Problem-Solving Framework

This framework applies to every problem in this course and every problem you will face in an interview. Skipping steps is the single most common reason freshers fail interviews on problems they technically know how to solve.

Step 1: Understand

Restate the problem in your own words. Confirm input format, output format, and constraints. Ask about edge cases: empty input, single element, duplicates, negative numbers, very large input.

Step 2: Identify the Pattern

Read the constraints. Map the input size to an acceptable complexity class. Match the problem description to a known pattern. This is the core skill the entire course builds.

Step 3: Plan

Write 3-5 lines of pseudocode or describe the approach verbally. Walk through one small example by hand to verify logic before writing code.

Step 4: Implement

Write clean code with clear variable names. Handle edge cases at the top of the function. Follow your plan from Step 3. Do not invent a new approach mid-implementation.

Step 5: Verify

Trace through your code with the example from Step 3. Test at least one edge case. State the time and space complexity unprompted.

Step	What You Do	Time in Interview
1. Understand	Restate, clarify, edge cases	~1 min
2. Identify Pattern	Constraints to complexity, signal to pattern	~1 min
3. Plan	Pseudocode, walk through example	~2-3 min
4. Implement	Clean code, narrate while coding	~15-20 min
5. Verify	Trace, test edge case, state complexity	~3-5 min

Interview Tip: Steps 1 through 3 should take 5-7 minutes. This feels slow, but interviewers reward it. Jumping to code in the first 30 seconds signals that you do not think before you build.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 2: Constraints Tell You the Pattern

The input size constraint in a problem is not decoration. It directly tells you which complexity classes are acceptable, which narrows the set of patterns that can work.

Input Size to Acceptable Complexity

Input Size (n)	Max Acceptable Complexity	What This Allows
n <= 20	O(2^n)	Backtracking, bitmask enumeration
n <= 500	O(n^3)	Triple nested loops (rare in interviews)
n <= 10^4	O(n^2)	Nested loops, brute force pair checking
n <= 10^6	O(n log n) or O(n)	Sorting, hash maps, sliding window, two pointers
n > 10^7	O(log n) or O(1)	Binary search, math, bit manipulation

Pattern Recognition Signals (Starter Map)

This table grows with every section. By the end of the course, this map becomes your primary problem-solving tool.

Problem Signal	First Pattern to Try	Section
"Find pair summing to target"	Hash map or two pointers (sorted)	5, 6
"Contiguous subarray with condition"	Sliding window	7
"Sorted input, find element"	Binary search	8
"Maximum subarray sum"	Kadane's algorithm	3
"Check existence / count occurrences"	Hash set or hash map	5
"Matching brackets / nesting"	Stack	10
"Next greater / next smaller element"	Monotonic stack	10
"Cycle detection / find middle"	Fast-slow pointers	11
"Generate all permutations / subsets"	Backtracking	12
"Locally optimal, never revisit"	Greedy	13
"Unique element among duplicates"	XOR	14
"Tree depth / height / validate BST"	Recursive tree patterns	15
"Connected components / shortest path (unweighted)"	BFS or DFS	16
"Count ways / minimize cost / overlapping subproblems"	Dynamic programming	17

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 3: How to Communicate: Brute Force to Optimal

Technical interviews are conversations, not exams. The interviewer needs to hear your reasoning. If you code silently for 20 minutes, they have zero evidence of your process, cannot give partial credit, and cannot nudge you in the right direction.

Always State the Brute Force First

Even if you immediately see the optimal solution, state the brute force. It proves you understand the problem space, gives a working baseline if you get stuck on the optimal, and creates the improvement narrative interviewers are trained to look for.

The Verbal Structure

Phase	Time	What You Say
Restate the problem	~30 sec	"So I'm given [input] and I need to return [output]. Can the input be empty? Can there be duplicates?"
Brute force, then optimal	~1-2 min	"The brute force checks every pair with nested loops, O(n^2). I can improve this with a hash map for O(n) lookups, giving O(n) time and O(n) space."
Confirm with interviewer	~10 sec	"Should I go ahead and code the hash map approach?"
Code while narrating	~15-20 min	"I'm iterating through the array. For each element, I check if the complement exists in the map..."
Test and discuss	~3-5 min	"Let me trace through this example. At index 0... Time is O(n), space is O(n)."

Interviewer Follow-Up Signals

What They Say	What It Means
"Can you do better?"	A solution one complexity class lower exists
"Can you do this in one pass?"	O(n) is achievable

What They Say	What It Means
"What if the array is sorted?"	Two pointers or binary search applies
"What if memory is not a concern?"	Trade space for time with a hash map
"What about edge cases?"	Test empty input, single element, all duplicates

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 4: Section Cheat Sheet

The Five-Step Framework

Step	Action	Key Deliverable
Understand	Restate, clarify, edge cases	Confirmed problem definition
Identify Pattern	Constraints to complexity, signal to pattern	Named pattern and expected complexity
Plan	Pseudocode, walk through example	Verified approach before coding
Implement	Clean code, edge cases first, narrate	Working solution
Verify	Trace example, test edge case, state complexity	Time and space stated unprompted

Constraint-to-Complexity Quick Reference

n	Target	Typical Patterns
<= 20	O(2^n) or O(n!)	Backtracking, brute force
<= 10^4	O(n^2)	Nested loops
<= 10^6	O(n log n) or O(n)	Sorting, hash maps, two pointers, sliding window
> 10^7	O(log n) or O(1)	Binary search, math, bit tricks

Pattern Recognition Starter Map

Signal	Pattern
"pair summing to target"	Hash map (S5) or two pointers on sorted (S6)

Signal	Pattern
"contiguous subarray/substring with condition"	Sliding window (S7)
"sorted array, find element"	Binary search (S8)
"matching/nesting"	Stack (S10)
"generate all possibilities"	Backtracking (S12)
"locally optimal, never revisit"	Greedy (S13)
"tree depth/height/validate"	Recursive tree patterns (S15)
"components/shortest path"	BFS/DFS (S16)
"count ways/minimize/overlapping subproblems"	DP (S17)

Verbal Answer Structure

Restate (30 sec), brute force then optimal with complexity (1-2 min), confirm approach (10 sec), code while narrating (15-20 min), test and state complexity (3-5 min).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina